EMT admin console: BFF auth plan

Reusing the personal-blog BFF auth implementation for the EMT sys-admin console. Analysis only. No BFF code has been changed.

Scope

The BFF's job is narrow: log the user in through EMT identity, keep the access token in an HttpOnly cookie, and attach it as a bearer token on calls to the .NET APIs. The .NET services do the real authorization. Reference files from personal-blog-site: `utils/httpClientUtil.ts`, `utils/authUtil.ts`, `utils/serverApi.ts`, `app/api/auth/me/route.ts`, and the login, logout and refresh routes.

1. Reuse as-is

These carry over with no design change.

Blog	EMT location	Notes
`httpClientUtil.ts` (createHttpClient)	`lib/data/client.ts`	Axios with a base URL, a timeout, and the self-signed-certificate agent outside production.
`serverApi.ts` getAuthHeaders()	`lib/data/client.ts`	Reads access_token from cookies() and turns it into Authorization: Bearer.
Login route: forward identity's Set-Cookie	Login Server Action	Tokens stay HttpOnly; browser code never sees them.
Logout route: delete access_token and refresh_token	Logout Server Action	Still needed, because EMT identity's logout doesn't clear cookies.
`normalizeUser()`	`lib/data/identity/auth.ts`	Keeps SecurityStamp and the rest of IdentityUserDTO out of the UI.
Only one refresh request at a time	proxy.ts (section 4)	More important in EMT, because its refresh tokens are single-use.

2. Tweak for EMT

Small changes to fit EMT's setup.

Blog	Tweak for EMT
One BACKEND_URL	Two backends: `IDENTITY_URL` and `EMT_CORE_URL`, chosen by `createHttpClient("identity" | "emtCore")`.
`getInitialUser()` in the root layout	Call it in `(dashboard)/layout.tsx` so the login page doesn't load the user, and redirect to /login when it returns null. It replaces the placeholder getCurrentUser(). See section 5 for the cache() optimization.
normalizeUser defaults the role to "user"	Use the real role from /me, since EMT only allows sys-admin.
AuthProvider client context	Probably not needed: the user is passed to the sidebar as a prop. Add it later only if deeper browser components need the user.

Blog	Tweak for EMT
`app/api/*` route handlers	Server Actions, as AGENTS.md asks for. Same logic, no extra HTTP hop.
Login sends userName	The EMT form says Email. Align the form label or identity's login field.

3. Improve

Gaps in the blog that matter more in EMT.

Gap	Improvement
Refresh only happens through the browser's axios interceptor	EMT loads most data in Server Components, which skip the interceptor. Refresh in `proxy.ts` instead (section 4). Without this, sessions effectively end after 30 minutes.
No route protection	`proxy.ts` sends users without a session to /login. The role check stays with the .NET services.
serverApi turns errors into [] or null	Throw and show an error page instead. Otherwise a backend outage looks like "no tenants".
Identity has no /me endpoint	Done: identity serves `GET sys-api/auth/admin/me`, returning id, userName, email, firstName, lastName, role and isActive from the joined identity and domain user.

4. Server-side refresh in proxy.ts

How sessions stay alive without the browser interceptor.

The two tokens expire at different times: the access_token cookie after 30 minutes, the refresh_token cookie after 7 days. Something has to swap them, and in Next only proxy.ts both runs before every page render and Server Action *and* can set cookies. Browser code can't read the HttpOnly cookies, and Server Components can't set cookies.

Request arrives with	proxy.ts does
Access token valid for more than 1 minute	Nothing. The request continues.
Access token missing (cookie expired) or under 1 minute left, and a refresh token	Refreshes through identity, puts the new tokens on the request (so this render uses them) and on the response (so the browser keeps them), then continues. Prefetches skip this.
No usable session, refresh rejected (401)	Clears both cookies; redirects to /login (or shows it).
No usable session, identity unreachable	Keeps the cookies; redirects to /login (or shows it).

Who decides what: the proxy only reads the token's exp claim, a base64-decoded JWT body with no signature check, to decide *when* to refresh. It never decides whether a user is signed in. Identity's /me does that, in two places: the dashboard layout (no user: redirect to /login) and the login page (user: redirect to /). Because both use /me, a token that hasn't expired but is rejected (user deleted, key changed) can't cause a redirect loop between /login and /.

Concurrent refreshes: refresh tokens are single-use. An in-memory map of in-flight refreshes makes overlapping requests share one refresh instead of each spending the token. This covers a single frontend instance; with several, add a short reuse window in identity.

Not in the BFF: signature verification. Tokens are signed with the shared Jwt:Key (HS256), which can also mint tokens, so it must never be in this repo. If local verification is ever needed, switch identity to asymmetric signing (RS256/ES256) with a public JWKS endpoint.

`proxy.ts` (project root)

```
import { NextResponse, type NextRequest } from "next/server";
import { refreshSession, type RefreshResult, type TokenCookie } from "@/lib/data/identity/auth";

const REFRESH_EARLY_MS = 60_000; // refresh when under 1 minute left

// One refresh per refresh token at a time; overlapping requests wait for the same one.
const inFlight = new Map<string, Promise<RefreshResult>>();

// Reads exp from the JWT body. No signature check: only decides *when* to refresh.
function expiresAt(token: string): number {
  try {
    const payload = JSON.parse(Buffer.from(token.split(".")[1], "base64url").toString());
    return payload.exp * 1000;
  } catch {
    return 0;
  }
}

function refreshOnce(token: string): Promise<RefreshResult> {
  let pending = inFlight.get(token);
  if (!pending) {
    pending = refreshSession(token).finally(() => inFlight.delete(token));
    inFlight.set(token, pending);
  }
  return pending;
}

function withCookies(response: NextResponse, cookies: TokenCookie[]) {
  cookies.forEach((c) => response.cookies.set(c.name, c.value, c.options));
  return response;
}

export async function proxy(request: NextRequest) {
  const isLoginPage = request.nextUrl.pathname === "/login";
  const access = request.cookies.get("access_token")?.value;
  const refresh = request.cookies.get("refresh_token")?.value;
  let result: RefreshResult | undefined;

  // 1. Access token valid for more than a minute: continue.
  //    (/login decides for itself, via /me, whether to send a signed-in user to "/".)
  if (access && expiresAt(access) > Date.now() + REFRESH_EARLY_MS) {
    return NextResponse.next();
  }

  // 2. Access token missing or expiring, but a refresh token: refresh (not for prefetches).
  if (refresh) {
    if (request.headers.has("next-router-prefetch")) return NextResponse.next();

    result = await refreshOnce(refresh);
    if (result.status === "ok") {
      result.cookies.forEach((c) => request.cookies.set(c.name, c.value)); // this render sees new token
      return withCookies(NextResponse.next({ request }), result.cookies);  // browser keeps new token
    }
  }

  // 3. No usable session: show /login, send everything else there.
  const response = isLoginPage
    ? NextResponse.next()
    : NextResponse.redirect(new URL("/login", request.url));

  // Identity rejected the refresh token: clear the dead cookies.
  // (If identity was just unreachable, keep them so the session works once it's back.)
  if (result?.status === "invalid") {
    response.cookies.delete("access_token");
```

```
    response.cookies.delete("refresh_token");
  }
  return response;
}

// Skip Next's build files, image optimizer, and anything with a file extension (public/ assets).
export const config = {
  matcher: ["/((?!_next/static|_next/image|.*\\..*).*)"],
};
```

`lib/data/identity/auth.ts`: additions used by proxy.ts and login()

```
export type TokenCookie = {
  name: string;
  value: string;
  options: { httpOnly: true; secure: boolean; sameSite: "lax"; path: "/"; expires?: Date };
};

export type RefreshResult =
  | { status: "ok"; cookies: TokenCookie[] }
  | { status: "invalid" } // identity rejected the refresh token (401)
  | { status: "error" };  // identity unreachable / 5xx

const TOKEN_COOKIES = ["access_token", "refresh_token"];

// Keeps only the two token cookies from identity's Set-Cookie headers. Also used by login().
export function parseTokenCookies(setCookie: string[] = []): TokenCookie[] {
  return setCookie.flatMap((header) => {
    const [pair, ...attributes] = header.split(";");
    const [name, value] = pair.trim().split("=");
    if (!TOKEN_COOKIES.includes(name)) return [];

    const expires = attributes
      .map((a) => a.trim())
      .find((a) => a.toLowerCase().startsWith("expires="))
      ?.slice("expires=".length);

    return [{
      name,
      value,
      options: {
        httpOnly: true,
        secure: process.env.NODE_ENV === "production",
        sameSite: "lax",
        path: "/",
        expires: expires ? new Date(expires) : undefined,
      },
    }];
  });
}

// Identity reads the refresh token from its cookie, so send it as one.
export async function refreshSession(refreshToken: string): Promise<RefreshResult> {
  try {
    const response = await createHttpClient("identity").post("/api/auth/refresh", null, {
      headers: { Cookie: `refresh_token=${refreshToken}` },
    });
    return { status: "ok", cookies: parseTokenCookies(response.headers["set-cookie"]) };
  } catch (error) {
    const invalid = axios.isAxiosError(error) && error.response?.status === 401;
    return { status: invalid ? "invalid" : "error" };
  }
}
```

Login page and dashboard layout: the /me checks

```
// app/login/page.tsx: signed-in users skip the form (decided by /me, not the proxy)
const user = await getInitialUser();
if (user) redirect("/");
```

```
return <LoginView />;

// app/(dashboard)/layout.tsx: no session -> login
const user = await getInitialUser();
if (!user) redirect("/login");
```

Verify when implementing: auth.ts imports server-only and, through client.ts, next/headers. proxy.ts runs in a separate server bundle; check these load there (start the dev server and open /login). If not, move refreshSession and parseTokenCookies into their own file.

5. Optimization: cache() on getInitialUser

Optional. An addition on top of the blog, not a fix.

The blog calls getInitialUser() once, in the root layout. In EMT both the dashboard layout (sidebar) and the Overview page (welcome line, account card) need the user, and a Next layout can't pass data to its page as props. Without caching, every Overview load makes two /me calls.

`export const getInitialUser = cache(async (): Promise<User | null> => { ... });`

Aspect	Detail
Benefit	One /me call per request, however many components ask for the user.
Cost	One wrapper from React itself. No new dependency.
Safety	The result lives only for that one server request, so one user's data is never served to another.
If skipped	Works the same; Overview just makes one extra /me call.

6. Login: Server Action

Decision: the BFF login is a Server Action, not an app/api/auth/login route.

It's still the BFF, just a different way to reach it. Next's own App Router authentication guide uses a form with a Server Action and useActionState; the fetch('/api/auth/login') route is its Pages Router example. The only caller is the login form, and the backends are already testable directly against the deployed services, so a public BFF login URL adds nothing. Axios is still used, on the server, inside login(). Route handlers stay the tool for anything that needs a real URL: webhooks, OAuth callbacks, other clients, downloads.

`lib/data/identity/auth.ts`: add login()

```
import { cookies } from "next/headers";
// ...existing imports: axios, createHttpClient, parseTokenCookies (section 4), etc.

// Logs in through identity and copies its token cookies onto our own domain.
// Returns false for bad credentials; throws for anything else (e.g. identity down).
export async function login(userName: string, password: string): Promise<boolean> {
  try {
    const response = await createHttpClient("identity").post("/api/auth/login", {
      userName,
      password,
    });

    const cookieStore = await cookies();
    parseTokenCookies(response.headers["set-cookie"]).forEach((c) =>
      cookieStore.set(c.name, c.value, c.options),
    );
    return true;
  } catch (error) {
```

```
    if (axios.isAxiosError(error) && error.response?.status === 401) return false;
    throw error;
  }
}
```

app/login/actions.ts

```
"use server";

import { redirect } from "next/navigation";
import { login } from "@/lib/data/identity/auth";

export type LoginState = { error: string | null };

export async function loginAction(_prev: LoginState, formData: FormData): Promise<LoginState> {
  const userName = String(formData.get("userName") ?? "").trim();
  const password = String(formData.get("password") ?? "");

  if (!userName || !password) {
    return { error: "Enter your username and password." };
  }

  try {
    const ok = await login(userName, password);
    if (!ok) return { error: "Username or password is incorrect." };
  } catch {
    return { error: "Couldn't reach the sign-in service. Try again." };
  }

  redirect("/"); // must be outside try/catch: redirect() works by throwing
}
```

Difference from PB	Why
Re-sets each cookie with cookies().set(...) instead of forwarding raw Set-Cookie headers	A Server Action has no response object to append headers to. It also lets the BFF keep only the two token cookies, skip identity's .AspNetCore.Identity.Application cookie, and use SameSite=Lax.
Copies each cookie's Expires from identity	The cookie still disappears when the token expires, which the proxy.ts refresh relies on.
401 and other errors give different messages	Bad credentials vs identity unreachable. The raw error never reaches the browser.

Form changes: the login page is now /login. In LoginForm the email field becomes name="userName" labelled Username (identity's login takes a username), and the form uses useActionState(loginAction, { error: null }) with <form action={formAction}>.

Testing: Playwright is unaffected; it drives the real form. In Jest, the action is a plain async function: mock @/lib/data/identity/auth and next/navigation, then call it with a FormData.

Summary

About 70% is reused as-is. Most of the tweaks are config and file locations. The one improvement that needs real design is the server-side refresh in `proxy.ts` (section 4). The identity `/me` prerequisite is done: identity now serves `GET sys-api/auth/admin/me`.